

An agent-based multiservice negotiation for eCommerce

W Merlat

Future generations of electronic commerce infrastructures will facilitate multiservice negotiations — negotiations between one customer and several providers regarding the provision of a personalised bundle of services. These negotiations allow the customer to obtain, for example, a complete travel package made up of several flights and hotel reservations, and matching the customer's specific requirements (regarding budget for example).

The main issue in a multiservice negotiation is to handle the constraints that tie together the services within the bundle. Another is to design a scalable negotiation protocol, i.e. one that performs well regardless of the size of the bundle of services.

A specific multiservice negotiation is proposed that has two distinctive properties. Firstly, its protocol is defined by a dynamic and decentralised constraint satisfaction algorithm. Secondly, the protocol is embedded within a set of mobile software agents, which perform the negotiation on behalf of the customer. The negotiation is tested on a travel agency application, and exhibits several valuable qualities. It is more scalable than traditional centralised facilitator-based solutions. It can also be customised easily (for optimisation purposes for example), without upgrading software on the providers' sides. Additionally, it is very suitable for mobile users, who are intermittently connected to the network.

1. Introduction

Electronic commerce (eCommerce) is a rapidly growing area of interest to telecommunications companies. BT, among others, aims at providing platforms and services for facilitating interactions between customers and providers (people or organisations) [1].

Currently, most facilitation services enable one-to-one interactions, which take place between one customer and one provider. BT Trading Places is of this kind [2]. From a customer's perspective, it facilitates the search for a provider, according to a set of requirements expressed by the customer (type of product, price range, etc). It then facilitates the negotiation between the customer and the provider, and monitors the actual supply of the product. With one-to-one negotiations, the only customer needs that can be satisfied are those that directly match the services offered by providers.

This paper investigates multiservice negotiations, or one-to-*N* negotiations. They involve many service providers and one customer¹. In a multiservice negotiation, the customer obtains a bundle of services from different providers. The services are mutually constrained by the customer's requirements (for example, time constraints tying two consecutive flight reservations in a journey). Consequently they cannot be obtained by a number of one-to-one independent negotiations. A multiservice negotiation

can be seen as a set of one-to-one negotiations running concurrently and in a co-ordinated manner, so as to obtain a bundle of mutually compatible services.

By enabling the composition of value-added services out of (more basic) services, this feature can match customer needs that could not be matched otherwise. It is therefore expected to play a key role in future eCommerce infrastructures.

This paper proposes a general multiservice negotiation procedure (valid for any kind of service). The procedure itself is performed by a set of mobile software agents acting on behalf of the customer. They search the network for appropriate providers, and negotiate with them for a bundle of services. They can also resume the negotiation and adapt the bundle following the failure of a service (e.g. flight delay or cancellation).

In the remainder of the paper, the multiservice negotiations are explained in more detail. Then the current approaches to multiservice negotiations and mobile agent technology are reviewed. The case study for this work (the travel agency application) is then introduced. The travel

¹ Although this work is concerned with multiservice negotiations, it is very general and can be applied to any kind of commodity, for example manufactured products in a supply chain.

agency application domain is often used as a case study in this research area, where a customer initiates a multiservice negotiation in order to obtain a bundle of travel services (flight, hotel rooms) that compose a complete journey.

The field of distributed constraint satisfaction problems (DCSP) is described. Then a multi-service negotiation protocol derived from a DCSP algorithm is presented. Its implementation in Voyager is briefly described and analysed.

Although this paper uses an electronic retailing scenario (i.e. travel agency application), multiservice negotiations are relevant to various types of eCommerce transaction, be it negotiations between businesses as part of a supply chain or between customers and businesses in a retail context.

This paper provides a decentralised, agent-based approach to multiservice negotiation which is relevant to both, and can be viewed as a key enabling technology for eBusinesses of the future.

2. Multiservice negotiations

The function of a multiservice negotiation is firstly to find a bundle of services matching the customer's requirements, and secondly to adapt the bundle in the case of service failure. The negotiation ends when the customer no longer uses the services.

The services are mutually correlated by some compatibility constraints specified in the customer's requirements. The main issue of designing multiservice negotiation is the handling of the service correlation. This requires the definition of a complex 'one-to-N' negotiation protocol (one customer and multiple providers), since providers need to communicate together (whether directly or through an intermediary) in order to provide correlated services.

Multiservice negotiations are usually performed through a facilitator that acts as a unique intermediary between the customer and the providers. Because the facilitator acts as a bottle-neck (there is no direct provider-to-provider communications, all communications go via the facilitator), such an architecture is not scalable. Moreover, the way service correlation is handled is left unspecified.

It is proposed that several mobile agents be used as multiple and distributed intermediaries, thus removing any bottle-neck. Also, the negotiation protocol is inspired by techniques for distributed constraint satisfaction problems, which make explicit the way service correlation is handled.

3. Related works

3.1 Facilitator-based multiservice negotiations

Figure 1 illustrates the negotiation for a bundle of services, by the means of several 'mono-service' negotiations supervised by a central facilitator. The facilitator conducts each negotiation so as to enforce the mutual compatibility between services.

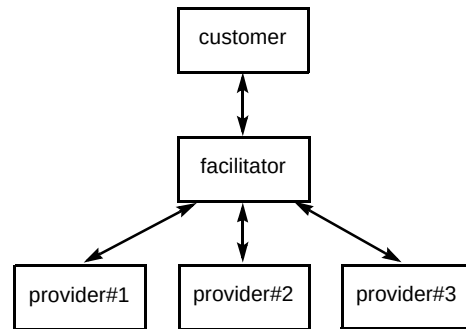


Fig 1 Multiservice negotiation via a facilitator (three providers involved).

A good example is the MOTIV (mobility and transport in intermodal traffic) project [3], which deals with the provision of travel-oriented services to customers. The facilitator is a travel broker, which initiates negotiations with several providers (one per service to obtain). Each follows a specific protocol, which includes the ability to modify the description of a service. This allows the travel broker to propagate modifications of one service to the other services of the bundle, in order to ensure that the customers' requirements constantly hold.

Because of its central role in such an architecture, the facilitator is a bottle-neck (as all messages are directed via the facilitator). Therefore it reduces system scalability.

Additionally, the way the facilitator manages customers' requirements, in particular, how it propagates modifications to services, is unspecified. This characteristic may be of importance when a large number of tightly constrained services are to be negotiated.

3.2 Mobile agents applications

Mobility denotes the ability of a software agent to roam wide area networks (such as the Internet), in order to interact with geographically dispersed servers and perform tasks on behalf of their owners [4]. The benefits of mobility are twofold:

- a potential reduction in network bandwidth consumption, as sending an agent to a remote host can be cheaper than initiating a remote session with that host,

- an enhanced tolerance to network failures — as with any software agent, mobile agents are autonomous and do not need a permanent connection with their owners while performing tasks at remote hosts, and therefore they can still work when the network is temporarily down and no communication is possible between the owners and the remote hosts.

Mobile agent technology is a rapidly evolving area of research. Several agent platforms currently provide support for mobility, such as Voyager [5], Aglets [6], etc. They all share the same general architecture based on places (Fig 2). Places host mobile agents and are located at different nodes of the network.

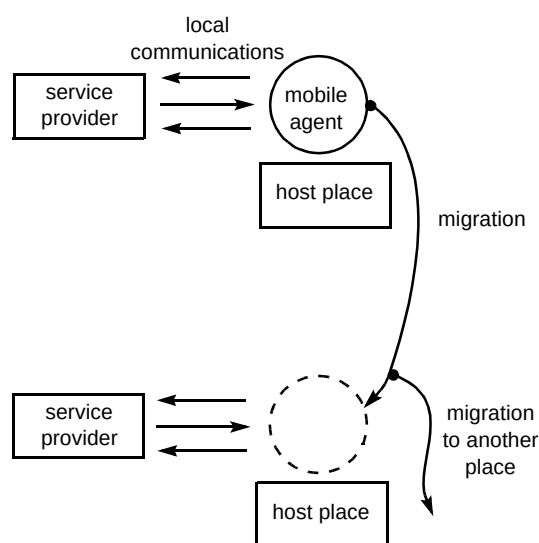


Fig 2 General architecture of mobile agent applications.

In the travel agency application (see section 4), places are located close to reservation systems of airline companies; and mobile agents move to a nearby place when they wish to negotiate with a particular airline company. Mobility can therefore reduce bandwidth consumption and enhance tolerance to network failures.

4. The travel agency application

4.1 Presentation

The travel agency application (TA) is a classical case study in the agent research area [7, 8]. It has been used for similar purposes by BT [9], France Telecom [10] and also by the German 'eCommerce-related' MOTIV consortium [3].

This TA system comprises a set of airline companies and hotels, and a customer. The customer asks for a bundle of services comprising travel tickets and hotel room reservations. Services are tied together by requirements expressed by the customer, who specifies the 'admissible' bundles of services that are acceptable.

Service failures may occur, causing the premature termination of the service. Failures can be caused by network and/or system breakdowns, or any internal problem occurring to the provider. In the TA example, flight delays and transport strikes are typical causes of failures. For the sake of continuity of service to the customer, the failing service has to be replaced by another one. During this process, some other services in the bundle may also have to be replaced in order to preserve the customer's requirements. Similarly, the bundle has to be modified if the customer changes their requirements — requesting new services (which then have to be incorporated into the bundle) and/or modifying requirements concerning existing services.

An analysis of the general type of 'travel agency application' is given in Ndumu et al [9], which identifies three issues:

- personalisation (the acquisition of customers' preference and requirements),
- pre-trip planning,
- post-plan monitoring.

While this work does not address the first issue, the two others are tackled by respectively building and adapting a bundle of services.

4.2 Service description

A service is described by a tuple of values — a type (represented by a string) and related parameters dependent on the type. Two types of services are used in the TA example:

- <'flight', origin, departure, destination, arrival, fare>,
- <'hotel', location, start, end, fare>.

The parameters 'departure', 'arrival', 'start' and 'end' are dates. They have continuous values and are represented in a standard format — '21 August 06.05pm'. The parameters 'origin', 'destination' and 'location' represent locations and have discreet values, represented as strings — 'Paris', 'London', 'NewYork', etc. The fare is a positive integer number.

4.3 Customer's requirements

Requirements are expressed as a set of constraints linking service descriptions. Two kinds of requirement can be distinguished:

- individual requirements tie the parameters of one single service — they concern the service regardless of the others in the bundle, and they are unary constraints on a service,

- collective requirements link parameters of different services — they are k -ary constraints ($k \geq 2$) that tie a subset of services in the bundle.

In the TA application, typical requirements set the maximum waiting time the customer will accept between two flights at an intermediary airport, the maximum price of the total bundle, etc. For example, a customer wishing to visit Paris from London from 21 to 23 August will express the following requirements (each line declares a constraint stating a particular requirement):

$\{(s_1, s_2, s_3) /$

$s_1 = \langle \text{'flight', 'London', *, 'Paris', *, *}, \rangle, \quad (c_1)$

$s_2 = \langle \text{'hotel', 'Paris', '21 August 07.00pm', '23 August 09.00am', *}, \rangle, \quad (c_2)$

$s_3 = \langle \text{'flight', 'Paris', *, 'London', *, *}, \rangle, \quad (c_3)$

$s_1.\text{arrival} < s_2.\text{start}, \quad (c_4)$

$s_2.\text{end} < s_3.\text{departure}, \quad (c_5)$

$s_1.\text{fare} + s_2.\text{fare} + s_3.\text{fare} < 1500 \} \quad (c_6)$

where $s_i.p$ refers to the parameter p of service s_i . “*” indicates that the value of the parameter is not constrained by the requirement. The corresponding constraint graph is shown in Fig 3.

The first three constraints (c_1 to c_3) state individual requirements on each service, and are used by mobile agents to pre-filter service proposals when issuing calls for proposals to providers (section 6.2). The three last state collective requirements — c_4 and c_5 state obvious schedule considerations, and c_6 sets a maximum price for the bundle (£1500).

5. Distributed constraint satisfaction problems and multiservice negotiation

Distributed constraint satisfaction problems (DCSP) can formalise distributed problem solving. A DCSP is defined as a set of distributed agents that communicate together in order to find a solution to the DCSP [11]. Each agent manages one variable of the constraint network — more specifically it selects a value for the variable that is compatible to the values of the other variables in the constraint network. To check for the mutual compatibility of their respective variables, agents communicate together, following a conversation protocol, which actually corresponds to the DCSP algorithm. Some DCSP algorithms include a repair procedure that modifies a solution following the the addition/removal of a constraint.

DCSP can be applied to multiservice negotiation. Variables are the services requested by the customer, and their possible values are the services offered by providers. To instantiate its associated variable, an agent buys a service from a provider. The repair procedure is used to adapt the solution following the failure of a provider. The failure amounts to the addition of a unary constraint that simply forbids the current value of the variable. The repair procedure finds a new solution that does not contain the forbidden value.

However, in DCSP algorithms, each agent knows the domain of its own variable (the set of all possible values for the variable). In contrast, in multiservice negotiations the domain for each variable is distributed among the set of providers (i.e. the information needed by each agent is distributed over the network). Therefore a multiservice negotiation protocol slightly differs from a DCSP algorithm (see section 6).

6. Multiservice negotiation with mobile agents

This section describes a multiservice negotiation using a DCSP algorithm and which is performed by a set of mobile agents.

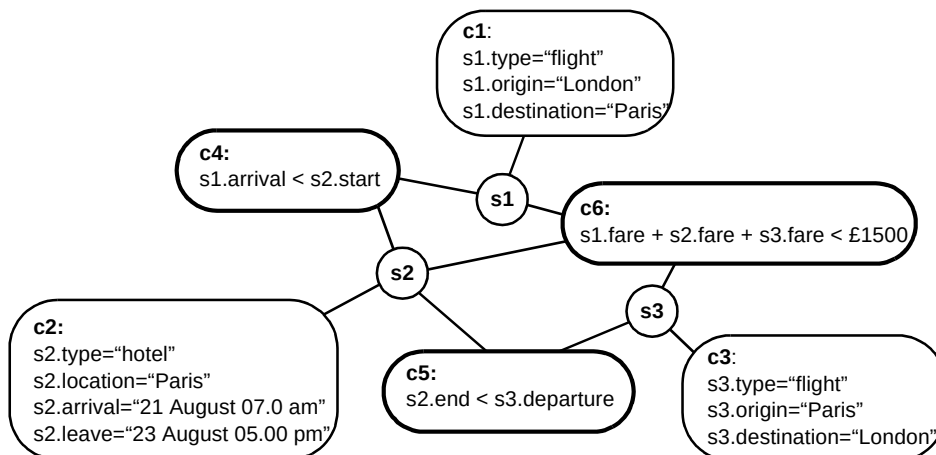


Fig 3 Constraint graph representing the customer's requirements.

The function of each mobile agent is to find one service of the bundle. Mobile agents communicate together so as to select services that are mutually compatible.

The conversation protocol follows a DCSP algorithm. However, to allow for the dispersion of information in the network, the negotiation starts with an initial phase where each mobile agent searches the network for possible services. After this phase, a standard DCSP algorithm is applied in combination with agent mobility. Agents communicate together and collectively select a bundle of services matching the customer's requirements. Each agent moves to (a place close to) the provider offering the service it has selected, and buys this service. Should a service fail, mobile agents resume communication (applying the repair procedure of the DCSP algorithm) in order to select a replacement bundle. Mobile agents then move to the providers involved in the new bundle to buy the corresponding services.

The exact negotiation protocols is composed of several phases, described in the following sections.

6.1 Building a hierarchy of mobile agents

At the initiation of the negotiation, the customer creates a set of mobile agents — one per service requested. Agents are ranked and form a linear hierarchy. The exact order is arbitrary, e.g. the one chosen in this work is related to the number of constraints linking the variable of each agent — the most constrained agents first. Two equally constrained agents are ranked in alphabetical order. Figure 4 shows the resulting hierarchy for the example given at section 4.3 (agents are noted a_i , constraints are noted c_j).

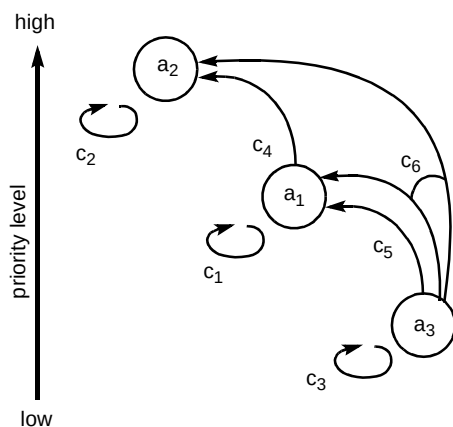


Fig 4 Hierarchy of agents for the TA scenario.

Each constraint is allocated to only one agent. A unary constraint is allocated to the agent managing the corresponding variable. A k -ary constraint ($k \geq 2$) is allocated to the agent that has the lowest priority of all agents tied by the constraint. Thus, each agent manages a set of constraints that links it to higher priority agents only.

6.2 Searching the network for service proposals

Each agent individually searches the network for available services matching the unary constraints it is given. Agents move to nodes hosting providers and issue calls for proposals. Providers answer a list of service descriptions, corresponding to the services they offer.

The specific TA implementation described in this paper uses a matchmaker (see section 7.1). However, as the precise search algorithm (e.g. exhaustive search of the network) is not relevant to this discussion, it will not be expanded upon here. Each agent stops when it has searched the entire network, or when it has found a maximum number of services (say, 20). At the end of this phase, each agent has retrieved a set of available services², and is located at the last node of the network it has visited.

6.3 Selecting a bundle of services

This phase is performed in a sequential way, following the hierarchy (Fig 5). Each agent, receiving a partial solution (the instantiation of the preceding variables) from the previous agent, instantiates its own variable by selecting one service proposal that matches the constraints it is allocated. If such a service proposal exists, then the agent appends it to the partial solution and passes it on to the next agent in the hierarchy. If no service proposals can satisfy the constraints, then the agent sends a backtracking message to the previous agent.

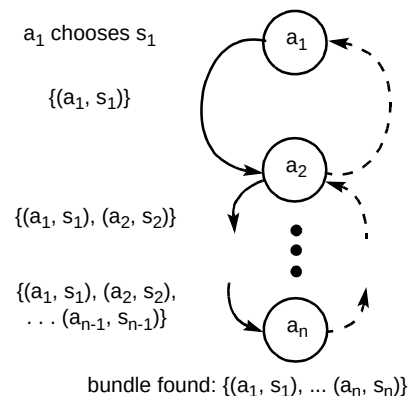


Fig 5 Selection of a bundle of services.

On receiving a backtracking message (dotted lines on Fig 5), an agent replaces the service proposal it had previously chosen by another one, and sends the new partial solution back on the next agent. If no such service proposal exists, then the agent forwards the backtracking message to the previous agent in the hierarchy. If the top agent in the hierarchy receives a backtracking message and cannot find a replacement service proposal, then the negotiation ends on a failure — no bundle of services could be found.

² For the sake of clarity, it is assumed here that all agents find at least one service proposal.

Once a bundle has been selected, each agent moves to the service provider that proposes the service it has chosen, and issues a 'request for provision' for that service. The multiservice negotiation is general and independent of the specific procedure used by mobile agents to obtain each service. This procedure can be a commercial transaction or a mere call to an API of the provider. It can use a protocol similar to the 'order of action' conversation in Volksen et al [3].

At this stage of the negotiation, a proper bundle of services is being provided to the customer. The negotiation then enters the final phase — maintaining the bundle.

6.4 Reacting to service failures

When a service failure occurs, the mobile agent that bought the service is notified by the provider. The mobile agent then initiates a repair procedure, which may replace other services in the bundle in order to preserve their mutual compatibility.

The exact procedure is as follows. The mobile agent tries to find a replacement service that is compatible with the current bundle among those it retrieved during the search phase. If such a service exists, the agent moves to the place of the corresponding provider and buys it — the bundle has adapted to the failure. If no such service exists then the agent sends a backtracking message to its ancestor in the hierarchy. The standard backtracking procedure follows and many services within the bundle may have to be replaced. The negotiation may end in a failure if no replacement bundle can be found.

6.5 Reacting to modifications of the customer's requirements

The general repair procedure allows for changes in the customer's requirements. During the negotiation, the customer may request another service, add or remove a requirement. In DCSP terms, this amounts to adding/removing constraints or variables. New mobile agents can be created during the negotiation and inserted into the hierarchy. Also, constraints can be added to or removed from mobile agents while the negotiation is running. However, space does not permit detailed discussion on that issue in this paper.

7. Experiments on the travel agency application

7.1 Computing infrastructure

The TA system is implemented using the Voyager platform [5]. The mobile agents are implemented as Voyager agents. The customer and service providers are also implemented as Voyager mobile agents, but they do not move. In a typical run, the system comprises about 50

service providers. On creation, each one randomly generates a set of 1000 services of the same type ('flight' or 'hotel') with random parameter values. The customer agent generates requests for arbitrarily long journeys, which are composed of 10 or more segments separated by hotel stays. Therefore, resulting multiservice negotiations involve dozens of mobile agents running simultaneously in the system.

A matchmaker was also built to facilitate the search by mobile agents. It is a simple yellow page server that returns a list of relevant provider addresses when given a service type. This matchmaker is unique within the network and is located at a special place known to all agents. Service providers register to the matchmaker at their creation, and give their address and the type of services they provide. When searching for providers, mobile agents move to the matchmaker's place and retrieve the list of addresses of all providers that have registered for the type of service they require ('flight' or 'hotel').

7.2 Snapshot of the negotiation

Figure 6 is a 'snapshot' of the negotiation and illustrates the configuration of the system, when agents a_1 , a_2 , a_3 respectively choose a service from AirFrance, Hilton and British Airways. Long arrows represent long distance messages between mobile agents, short arrows are local messages. The customer agent is represented to emphasise the fact that it is not actively involved in the negotiation. Host places for the mobile agents are not represented. The other service providers, airline companies and hotels, which are actually present in the network but currently not involved in the provision of a service, are not represented either. Also, for the sake of clarity, Fig 6 displays a configuration involving only three mobile agents. This configuration is to be compared to the solution with a facilitator (see section 5).

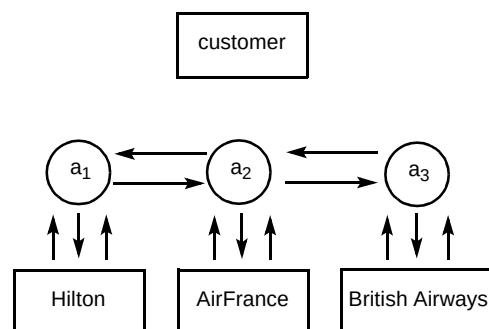


Fig 6 Configuration of agents for a particular bundle of services.

It should be emphasised that agents do not remain at the same location throughout the negotiation, since they may have to move when activating a replacement service in response to a service failure.

8. Analysis of the multiservice negotiation

8.1 Automatic bundle adaptation

It is important to note that the negotiation does not end when a bundle of services is found. The negotiation can resume at any moment (following the failure of a provider) in order to modify the bundle; any modification of a service is automatically propagated to the other services of the bundle, so that the customers' requirements constantly hold.

8.2 Local communications and asynchronous computing

As a direct result of the use of mobile agents, most of the negotiation is performed by the agents locally at servers' places. It is also performed asynchronously with regards to the customer, who is never directly involved in the negotiation.

8.3 Improved scalability

Calculations are distributed over the set of mobile agents, and each agent's processing is only a function of the number of constraints it is allocated, not of the number of agents (i.e. of the number of variables in the DCSP). Therefore the negotiation's performance declines gracefully as the size of the bundle (and consequently the number of agents) increases.

8.4 Flexibility of the negotiation protocol

The mobile agents act as negotiation intermediaries between the providers themselves, and also between the customer and the providers (compare Fig 6). They dynamically integrate the provider's information system in order to facilitate the multiservice negotiation. Therefore modifications to the negotiation protocol do not require any software upgrade on the providers' sides.

Similarly, various negotiation protocols can be provided in the form of different mobile agent classes. Indeed, various DCSP algorithms can be used to define different multi-service negotiation protocols, with distinctive qualities and weaknesses. On the initiation of the negotiation, it is up to the customer to select the negotiation protocol that is most suitable.

8.5 Suitability to mobile users

This negotiation procedure is particularly appropriate for mobile users who are only intermittently connected to the network, as they can disconnect from the network once the mobile agents have started roaming the network. The negotiation procedure is both cost-effective in terms of bandwidth consumption and battery power.

8.6 Integration with an eCommerce infrastructure

Although France Telecom has already experimented with an eCommerce platform based on the Telescript mobile agent technology [10], currently no operational eCommerce platform uses mobile agents.

Security concerns (related to the malicious behaviours of badly designed or hostile mobile agents) needs to be addressed before mobile agent technology can be used in commercial products. While this paper acknowledges this issue, it does not address it. Security for mobile agent systems is a very active research area [12].

Once such platforms exist, third party vendors can offer 'negotiation expertise' by way of mobile agent classes made available to customers (on a pay per use basis, for example). This also constitutes a business opportunity for telecommunications companies like BT, who could offer such services on the eCommerce infrastructures operated by their competitors.

9. Conclusions

This paper has described an investigation of the use of mobile agents to perform multiservice negotiations. The customer's requirements are expressed as a constraint network, and dynamic and decentralised constraint — satisfaction algorithms are used to specify decentralised negotiation protocols. Mobile agents actually perform the negotiation on behalf of the customer. Although in this case the travel agency application is used as a test scenario, the negotiation protocol itself is general and could easily be used in other circumstances.

Because of its decentralised nature, the negotiation procedure is scalable. It is also very suitable for mobile users, who are intermittently connected to the network.

Security issues need to be addressed before eCommerce infrastructures support mobile agents. Once this is done, the multiservice negotiation presented in this paper will allow eCommerce infrastructures to match customers' needs more closely by helping them to obtain personalised bundles of services.

Acknowledgements

This work was mainly carried out when the author was a PhD student at LIP6 (Laboratoire d'Informatique de Paris 6) of Paris 6 University in France. His PhD was funded by the French ONERA (Office National d'Etudes et de Recherches Aéronautiques).

Members of the Intelligent Business Systems Research Group and the Intelligent Systems Research Group from BT Adastral Park made useful comments during the reviewing of this paper. Jean-Pierre Briot and Min Jung Yoo from

LIP6 also supported the actual realisation of this work and the writing of this paper.

References

- 1 Special issue: 'Electronic commerce', BT Technol J, 17, No 3 (July 1999).
- 2 BT Trading Places Web site — <http://www.tradingplaces.bt.com>
- 3 Völksen G, Dietrich D and Steiner D: 'Intelligent agents for personal travel assistance', Chapter 2 in Hayzelden A and Bigham J (Eds): 'Software Agents for Future Communications Systems', Springer-Verlag (1997).
- 4 Nwana H S and Ndumu D T: 'A brief introduction to software agent technology', in Jennings N R and Wooldridge M J (Eds): 'Agent Technology: Foundations, Applications and Markets', Heidelberg, Springer-Verlag, pp 29—49 (1998).
- 5 'ObjectSpace', The Voyager Web site — <http://www.objectspace.cm>
- 6 Aglets Website — <http://www.trl.ibm.co.jp/aglets/index.html>
- 7 FIPA: 'Personal Travel Assistance', Draft Specification Part 4 (1997).
- 8 FIPA: 'Agent Management Support for Agent Mobility', — <http://drogo.cselt.stet.it/fipa/>
- 9 Ndumu D, Collins J C and Nwana H S: 'Towards desktop personal travel agents', BT Technol J, 16, No 3, pp 69—78 (July 1998).
- 10 Léger A and White J E: 'Towards intelligent telematics', Proceedings of CNET 50th anniversary Multimedia Forum, pp 55—62 (1995).
- 11 Yokoo M, Durfee E H, Ishida T and Kuwabara K: 'Distributed constraint satisfaction for formalizing distributed problem solving', in Proceedings of the 12th International Conference on Distributed Computing Systems, IEEE Computer Society Press (1992).
- 12 Farmer W M, Guttman J D and Swarup V: 'Security for mobile agents: issues and requirements', in Proceedings of the 19th National Information Systems Security Conference (NISSC96), pp 591—597 (1996).



Walter Merlat joined BT Intelligent Business Systems Research group in November 1997. Since then his work has focused on market-based control mechanisms for business processes and telecommunications networks.

He holds an MSc and a PhD in Artificial Intelligence from the Université Pierre et Marie Curie in Paris, France. He is also a Chartered Telecommunications Engineer of the French Institut National des Telecommunications.